

PROJECT REPORT

REINFORCEMENT LEARNING

Comparative Study of On-Policy (VPG) and Off-Policy (DQN) Algorithms

Application to the "Fruit Catcher" Environment

Author:

Hack GAZALIOU

Date:

March 2, 2026

Abstract

Reinforcement learning (RL) offers powerful solutions for solving sequential decision-making tasks where formulating explicit control rules proves complex. This is notably the case in the stochastic *Fruit Catcher* environment studied here, where an agent must simultaneously optimize target collection and the evasion of lethal threats. However, optimizing such policies frequently encounters issues of instability and convergence towards suboptimal local optima. This study proposes to solve this control problem while empirically comparing two fundamental algorithms: the *Deep Q-Network* (DQN) and the *Vanilla Policy Gradient* (VPG). Initial experiments demonstrate that transitioning from an observation space in absolute coordinates to an egocentric frame of reference (relative distances) successfully eradicates the agent's geometric positioning biases. The subsequent comparison of the two algorithms reveals a major trade-off between sample efficiency and stability. While the DQN displays a significantly faster initial learning rate, its *off-policy* architecture proves to be highly susceptible to instability and value overestimation in the presence of extreme penalties. Although the literature attributes superior theoretical potential to the DQN, the final results show a much more robust evasion policy under VPG. This empirical superiority of VPG thus highlights that, in the context of an initial implementation lacking advanced expertise in hyperparameter tuning, the intrinsic stability of *on-policy* methods constitutes a decisive practical advantage over algorithmic efficiency.

Keywords: Reinforcement Learning, Deep Q-Network, Policy Gradient, Feature Engineering.

Contents

1	Introduction	2
2	The Environment	2
3	Theoretical Framework	3
3.1	Types of Models	3
3.2	Vanilla Policy Gradient (VPG)	4
3.3	Deep Q-Network (DQN)	5
4	Engineering and Experiments (Logbook)	7
4.1	Phase 1: The Failure of Naive VPG	7
4.1.1	Neural Architecture and Initial Observation Space	7
4.1.2	Initial Configuration and Training	8
4.1.3	Architectural Improvements	10
4.2	Phase 2: Feature Engineering and Reward Shaping	11
4.2.1	First Victory: Escaping the Local Minimum and Behavioral Flaw	11
4.2.2	Attempted Corrections: Entropy and Data Augmentation	11
4.2.3	Paradigm Shift: From Absolute to Relative	12
4.2.4	The "Bomb Hunter" Anomaly	13
4.2.5	Separation of Streams: Synaptic Specialization	14
4.3	Phase 3: Transition to the Off-Policy Architecture: The Deep Q-Network (DQN)	15
4.3.1	Fundamental Differences and Internal Mechanics	15
4.3.2	Redefinition of Temporal Evaluation: Epochs vs Steps	16
4.3.3	Initial Configuration of the DQN Agent	16
4.3.4	Behavioral Analysis of DQN: Instability and Estimation Bias	17
4.3.5	Hyperparameter Adjustment: The Sensitivity of the Learning Rate	18
5	Results and Comparative Analysis	19
5.0.1	Performance Benchmark	19
6	Conclusion	19

1 Introduction

Reinforcement learning (RL) allows an agent to learn an optimal policy through trial and error and feedback on its performance.

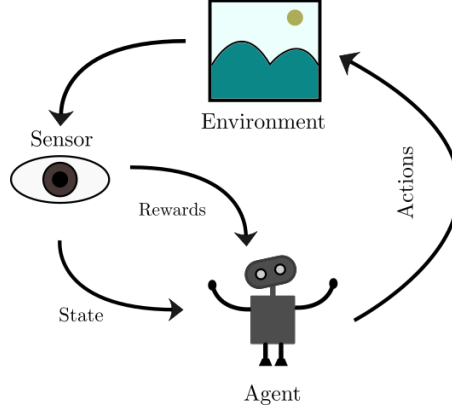


Figure 1: Schematic of the RL principle

In this project, we address a discrete control problem: the "Fruit Catcher" game. The difficulty lies in the need to arbitrate between collecting positive rewards (fruits) and avoiding critical penalties (bombs), all within an environment where the dynamics are fast and stochastic.

This report presents the implementation and comparison of two deep reinforcement learning architectures: the *Vanilla Policy Gradient* (VPG) and the *Deep Q-Network* (DQN). We will outline the different approaches employed (whether they failed or succeeded) to improve the agent's performance, as well as the feature engineering and reward shaping processes that allowed it to overcome local minima. Finally, we will analyze the performance gaps and the agent's behavioral differences between the two algorithms.

The objective of this study is twofold:

- **Engineering:** Design an agent capable of achieving advanced human-level scores.
- **Research:** Compare the efficiency of Policy-Based (VPG) and Value-Based (DQN) methods.

2 The Environment

The 'Fruit Catcher' environment is a discrete control simulation where the agent steers a receptacle (the basket) along a normalized horizontal axis. The objective is to intercept stochastic targets (apples, mangoes) generated from the top of the screen, while avoiding critical obstacles (bombs). The agent begins the episode with three lives, and the episode terminates when no lives remain. The rewards associated with the fruits were calibrated empirically, favoring mangoes, while any contact with a bomb results in the immediate end of the episode (terminal state).



Figure 2: Game environment

The temporal dynamics of the environment impose a strict reactivity constraint. An object falls vertically across the entire height of the screen in 334 time steps ($\Delta t \approx 5.57s$ at 60 FPS), while the agent can traverse the entire horizontal axis in 50 time steps ($\Delta t \approx 0.83s$) at maximum speed. The action space is discrete and has a cardinality of 5: $\mathcal{A} = \{L++, L+, \text{Stop}, R+, R++\}$. Fast displacements (± 0.02 units/step) and normal displacements (± 0.01 units/step) allow the agent to modulate its approach speed. The spawn frequency of objects as well as their probabilistic distribution are governed by the selected difficulty level. Table 1 details these parameters for the three available levels.

Table 1: Generation parameters according to the difficulty level

Level	Frequency (steps/fruit)	Probabilities		
		Apple	Mango	Bomb
Level 1 (Easy)	95	60 %	20 %	20 %
Level 2 (Medium)	65	50 %	25 %	25 %
Level 3 (Hard)	45	40 %	30 %	30 %

3 Theoretical Framework

3.1 Types of Models

One of the main dividing lines between the major families of reinforcement learning algorithms concerns the existence of an environment model. The central question is as follows: does the agent have (or learn) a model capable of predicting state transitions and rewards? In other words, can it anticipate what will happen after an action, or must it learn solely from direct experience?

Here, we focus on algorithms belonging to the so-called *model-free* family. The agent is therefore not capable of predicting the succession of environment states (i.e., it knows neither

when nor where the fruits will appear, and cannot predict their type).

The second distinction that can be made between RL algorithms is the separation between *value-based* and *policy-based* algorithms.

In *value-based* methods, the agent focuses on learning a value function that estimates the expected cumulative reward (return) of each state or state-action pair. The policy is established indirectly by selecting actions that maximize this estimated value.

Policy-based methods, on the other hand, rely on directly learning the policy without estimating a value function. The agent optimizes the policy by adjusting its parameters to maximize the expected return.

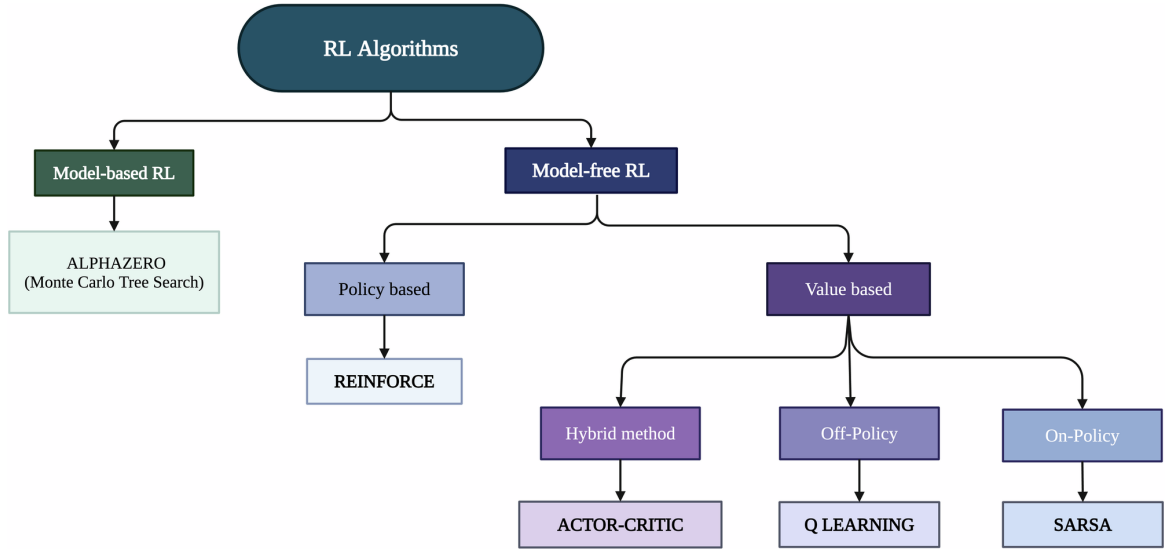


Figure 3: RL algorithm families

3.2 Vanilla Policy Gradient (VPG)

VPG belongs to the second family described above. It is one of the most primary forms of *policy-based* methods, meaning the algorithm directly optimizes the policy $\pi_\theta(a|s)$ by performing gradient ascent on the expected cumulative rewards.

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) A_t \right] \quad (1)$$

Here is the pseudo-code:

Algorithm 1 Vanilla Policy Gradient

-
- 1: **Input:** initial policy parameters θ_0
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: Collect a set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by executing the policy $\pi_k = \pi(\theta_k)$ in the environment.
 - 4: Calculate the cumulative future rewards (rewards-to-go) $\hat{R}_t$.
 - 5: Calculate the baseline b_k as the mean of the returns over the batch: $b_k = \frac{1}{N} \sum \hat{R}_t$.
 - 6: Calculate the advantage $\hat{A}_t$ (reward centering):

$$\hat{A}_t = \hat{R}_t - b_k$$

- 7: Estimate the policy gradient via:

$$\hat{g}_k = \frac{1}{|\mathcal{D}_k|} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) |_{\theta_k} \hat{A}_t.$$

- 8: Update the policy via gradient ascent:

$$\theta_{k+1} = \theta_k + \alpha_k \hat{g}_k,$$

- 9: **end for**
-

Here, the weights of the neural network defining the agent's policy are updated based solely on **the actions performed by the agent while using the policy in question**; this makes VPG *on-policy* (as are all *policy-based* methods in their standard form).

One of the main flaws of this type of method is that they "throw away" the collected data after each network weight update. The acquired experience only serves for the improvement immediately following it, and not for any subsequent ones. This leads to low sample efficiency.¹

3.3 Deep Q-Network (DQN)

The DQN, introduced by Mnih et al. (2015), learns a value function $Q(s, a)$ to estimate the quality of an action. It is therefore a *value-based* method. It is thanks to this algorithm that RL-trained agents were able to surpass humans in *Atari*-style games for the first time. The method introduces two major innovations to stabilize learning:

1. **Experience Replay:** A memory buffer to break temporal correlations.
2. **Target Network:** A stable target network to prevent the objective from moving during learning.

Unlike VPG, which seeks to maximize the expected return, the DQN seeks to minimize the Q-value prediction error (temporal difference error) with respect to a target. The loss function to be minimized is defined by the Bellman equation:

¹A good introduction to policy gradients is available [here](#)

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(\underbrace{r + \gamma \max_{a'} \hat{Q}(s', a'; \theta^-)}_{\text{Target}} - Q(s, a; \theta) \right)^2 \right] \quad (2)$$

Here is the pseudo-code, incorporating the *Experience Replay* mechanism:

Algorithm 2 Deep Q-Learning with Experience Replay

```

1: Initialization: Replay memory  $\mathcal{D}$  with capacity  $N$ 
2: Initialize the value function  $Q$  with random weights  $\theta$ 
3: Initialize the target value function  $\hat{Q}$  with weights  $\theta^- = \theta$ 
4: for episode = 1,  $M$  do
5:   Initialize the starting state  $s_1$ 
6:   for  $t = 1, T$  do
7:     Action Selection ( $\epsilon$ -greedy):
8:     With probability  $\epsilon$ , choose a random action  $a_t$ 
9:     Otherwise, choose  $a_t = \arg \max_a Q(s_t, a; \theta)$ 
10:    Execute action  $a_t$ , observe reward  $r_t$  and the next state  $s_{t+1}$ 
11:    Store transition  $(s_t, a_t, r_t, s_{t+1})$  in  $\mathcal{D}$ 
12:    Learning:
13:    Sample a random minibatch of transitions  $(s_j, a_j, r_j, s_{j+1})$  from  $\mathcal{D}$ 
14:    Calculate target  $y_j$ :
15:    if the episode terminates at  $j + 1$  then
16:       $y_j = r_j$ 
17:    else
18:       $y_j = r_j + \gamma \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$ 
19:    end if
20:    Perform a gradient descent step on  $(y_j - Q(s_j, a_j; \theta))^2$  with respect to  $\theta$ 
21:    Every  $C$  steps, update the target network:  $\hat{Q} = Q$ 
22:  end for
23: end for

```

Contrary to VPG, the DQN is an **Off-Policy** algorithm. This means it can learn from data generated by a different policy than the one it is currently optimizing (here, older data stored in the *replay buffer*).

This characteristic corrects the major flaw of VPG: sample efficiency. Each transition (s, a, r, s') can be reused multiple times for training, allowing the network to converge with far fewer interactions with the environment.

Trade-off between policy optimization and Q-learning: The primary advantage of policy gradient methods is their fundamentally coherent nature: they directly optimize the desired objective. This approach generally makes them more stable and reliable.²

Conversely, Q-learning methods only optimize the agent's performance indirectly, by training Q_θ to satisfy a self-consistency equation. This type of learning presents numerous potential failure modes, making it less stable in practice.³

²A good introduction to *Q-learning* is available [here](#).

³Instabilities can notably arise from function approximation and *bootstrapping*.

On the other hand, when Q-learning methods function correctly, they benefit from significantly higher sample efficiency, as they can reuse data much more effectively than policy optimization methods.

4 Engineering and Experiments (Logbook)

This section details the agent’s iterative evolution in the face of encountered failures.

4.1 Phase 1: The Failure of Naive VPG

4.1.1 Neural Architecture and Initial Observation Space

Throughout the study, the architecture of the neural network (the agent’s "brain") remained constant. It is a multilayer perceptron (MLP) consisting of two hidden layers of 64 neurons each, with ReLU activations.

Note: This dimensionality (64×64) is an empirical standard for low-dimensional control problems. It offers an ideal compromise between representational capacity (sufficient to approximate non-linear functions) and computational cost.

Initially, the observation space provided to the agent was designed to be as raw as possible, without manual *feature engineering*. The state vector S_t , of dimension 18, is structured as follows:

$$S_t = [x_{basket}, \underbrace{x_1, y_1, T_1, F_1}_{\text{Fruit 1}}, \dots, \underbrace{x_4, y_4, T_4, F_4}_{\text{Fruit 4}}, \text{Lives}] \quad (3)$$

4

Where for each fruit i :

- x_i, y_i : Normalized coordinates of the object.
- T_i : Object type encoding ($-1 = \text{Bomb}$, $0 = \text{Apple}$, $1 = \text{Mango}$).
- F_i : Binary presence flag (1 if the object exists, 0 otherwise).

The objective of this "naive" approach was to test the network’s abstraction capabilities: the agent was supposed to discover spatial and semantic relationships between these raw scalars on its own, much like human intuition, without being explicitly provided with notions of relative distance or speed.

⁴The number of lives is normalized; it represents the ratio between the remaining lives and the initial count.

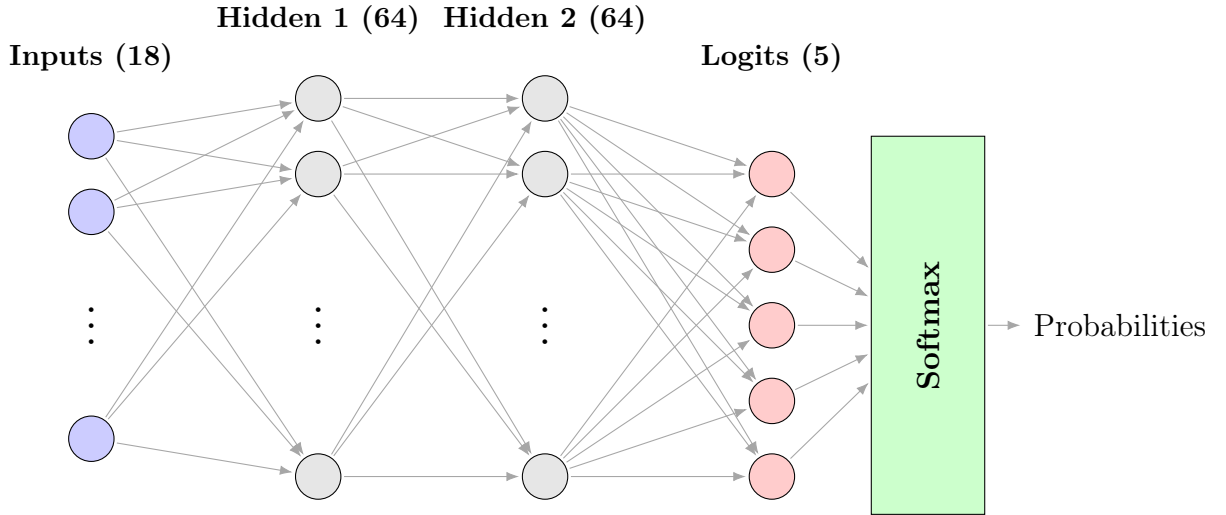


Figure 4: Architecture of the Multilayer Perceptron

4.1.2 Initial Configuration and Training

Before launching the first training run, uncertainty remained regarding the game level to choose for the learning phase. Had the agent been trained on a low level (1 or 2), the risk was that it would encounter the experience of a fruit very rarely during the episode, thereby creating a *reward density* issue. The vast majority of actions performed would have had no impact on the return (moving in empty space without contact), which would only have generated substantial noise in the gradient calculation. Conversely, it did not appear desirable to expose the agent exclusively to level 3 either, although this would have maximized reward density. Indeed, the initial fear was that at such a high spawn frequency, the agent would lose its lives too quickly, preventing it from accumulating useful experiences during the episode—an assumption that would later prove misleading.

Ultimately, it was decided to opt for a **domain randomization** approach. This consisted of randomly selecting the game level at the beginning of each episode. This allowed for a reduction in episode duration (now fixed at 500 steps) without sacrificing situation diversity.

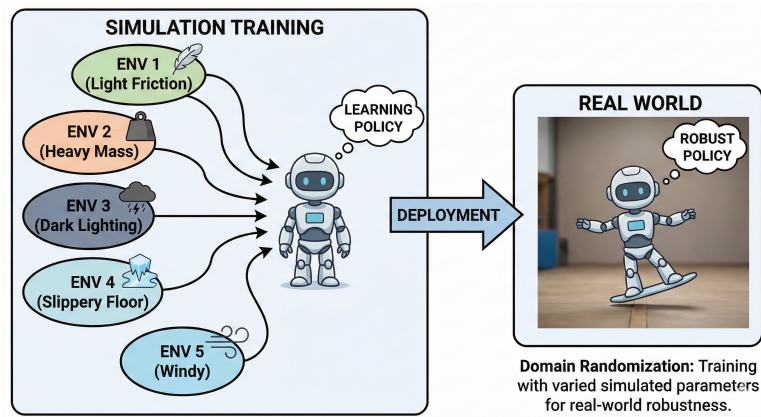


Figure 5: Schematic principle: Domain randomization

At the start of the project, lacking a strong intuition regarding the mechanics of an RL model, there was some concern that alternating between levels would hinder learning by sending a confusing signal that obscured the nature of the game. However, an analysis of the Markov

property (MDP) dispelled this doubt: since the agent’s decision $\pi(a|s)$ depends only on the present state and not the history, this variation does not "scramble" the learning. On the contrary, it acts as a regularizer, forcing the agent to generalize a broad behavioral law (reaction-movement) rather than overfitting to the specific dynamic of a single level.

Under this configuration, the initial training runs were a failure. After several sessions of 250 epochs consisting of 32 episodes each, the agent’s mean return per epoch did not evolve as it should have.

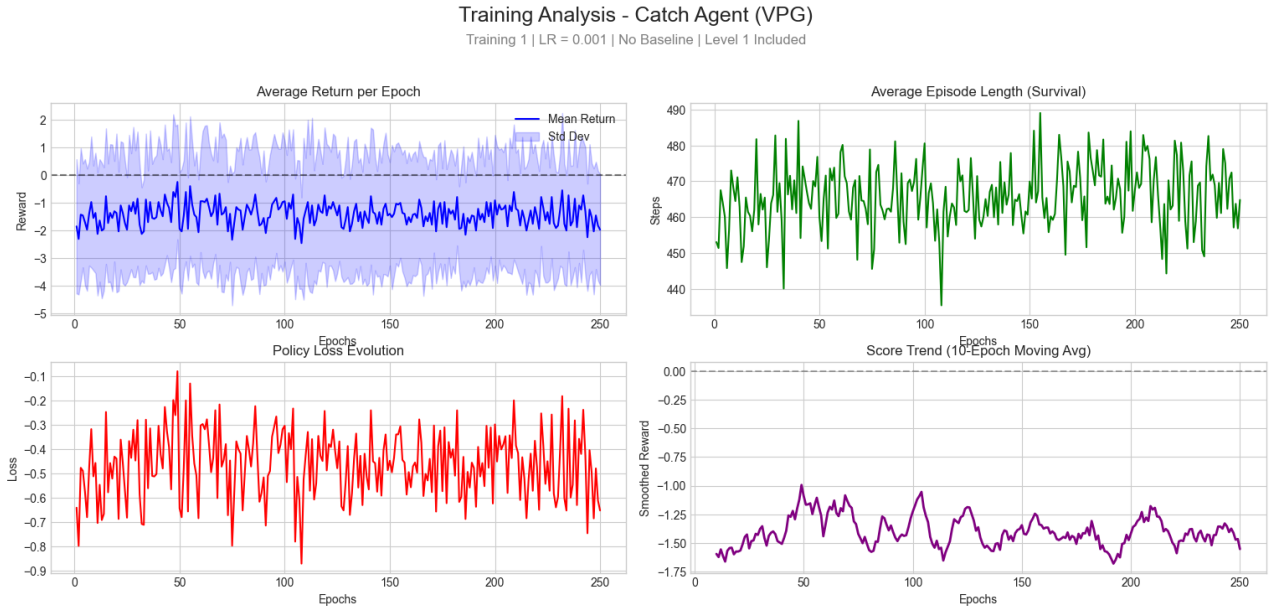


Figure 6: Results of training 1

Figure 6 presents four curves. The first is the most important for judging the success or failure of the training: it tracks the evolution of the mean return achieved by the agent at each epoch. An increase in this value indicates that the agent’s policy is evolving in a way that earns it more points. Here, we observe that this quantity merely oscillates around a negative value.

The second curve represents the average duration of the episodes at each epoch. An increase in this metric would signify that the agent is managing to survive longer before losing all its lives. Here, it only evolves stochastically around a mean value nearing 470 steps.

It is important to note that in the present configuration, this curve provides little information. Indeed, the agent simply could not die before approximately 400 game steps, as the game was initialized with an empty screen. As explained previously, the first objects could not reach it before the 334th step. Since bomb spawns were rare, most episode terminations were due to the 500-step hard limit. This analytical bias will be corrected in subsequent iterations of the study.

The third curve represents the evolution of the loss function. Its interpretation in reinforcement learning (particularly for value-based methods) is quite different from its conventional meaning in classical machine learning. We will detail this later in the report.

Finally, the fourth curve depicts the moving average of the return over 10 epochs. It helps smooth out the noise inherent in the stochastic environment to better visualize global learning trends.

4.1.3 Architectural Improvements

Faced with these disappointing initial results, several iterations were conducted to correct the structural weaknesses of the training. Firstly, the implementation of a *baseline* (normalizing rewards to zero mean and unit variance) helped reduce noise in the gradient. This modification forces the agent to evaluate its actions relative to the episode average rather than in absolute terms.

Next, to correct a bias induced by the scalar encoding of objects (which falsely led the network to assume that an apple (0) was the exact midpoint between a mango (1) and a bomb (−1)) a *one-hot* encoding scheme was adopted. Finally, to address a sample efficiency problem caused by the agent spending 60% of the episode alone on screen before the first object arrived, the environment was modified to pre-populate the upper section of the play area from the very first time step.

These improvements were tested in tandem with various learning rate values, yet failed to generate a significant increase in scores. Nonetheless, these experiments proved analytically crucial: the decrease and stabilization of the loss function highlighted that the network was not incapable of learning, but that it was certainly converging toward a suboptimal behavior.

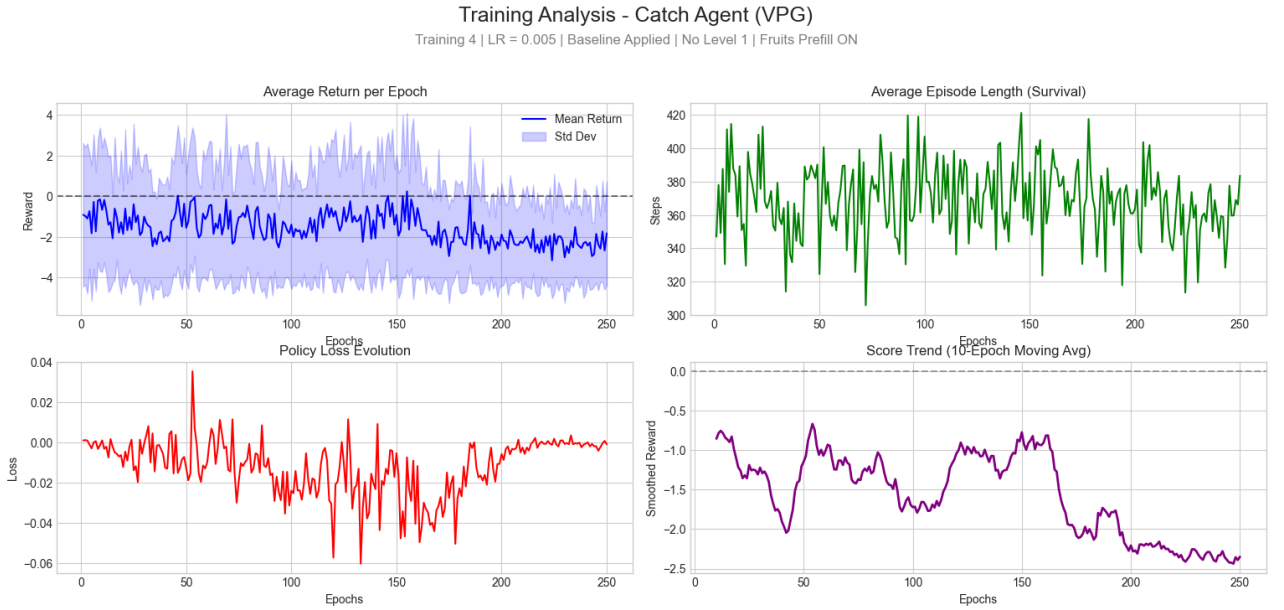


Figure 7: Results of training 4

The third curve in Figure 7 reveals that the loss function values are on the order of 10^{-2} compared to 10^{-1} in Figure 6.

The algorithm had in reality suffered a *policy collapse*. The agent had mathematically deduced that the risk of striking a bomb (−3 points, immediate death) outweighed the benefit of catching a fruit (maximum +2 points). Consequently, the network converged on a local strategy of "cowardice," systematically taking refuge in a corner of the screen to minimize risk-taking.

4.2 Phase 2: Feature Engineering and Reward Shaping

4.2.1 First Victory: Escaping the Local Minimum and Behavioral Flaw

Following a drastic redefinition of the reward function (+8 for a mango, +5 for an apple, -3 for a bomb), coupled with ϵ -greedy exploration, the training finally crossed a decisive milestone.

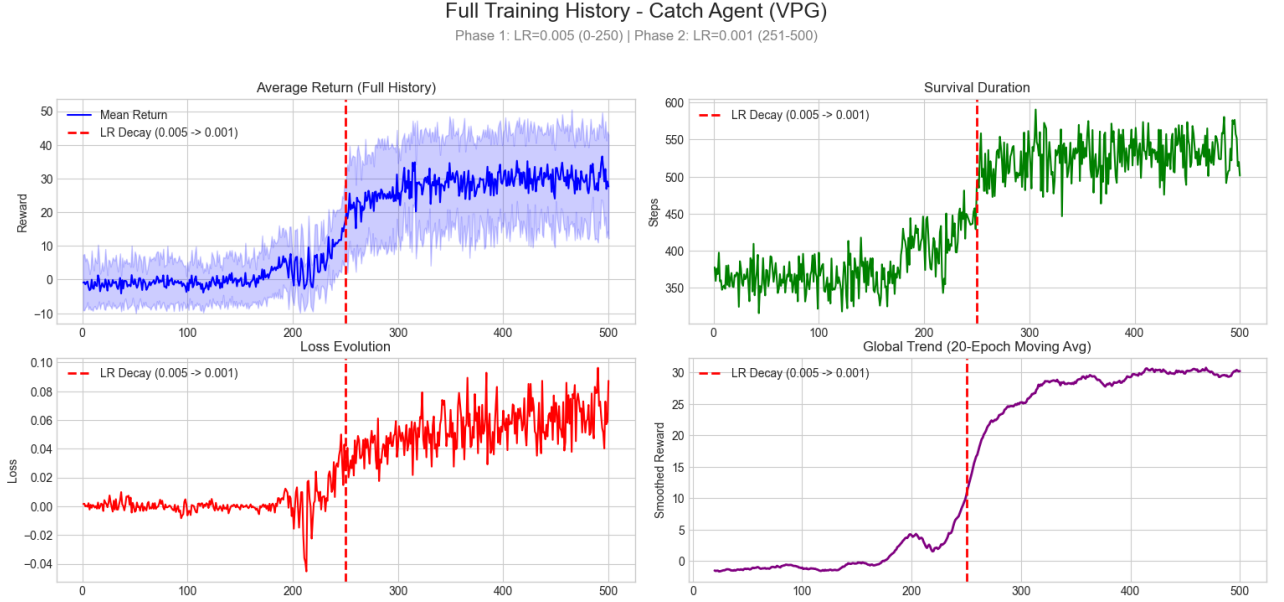


Figure 8: Results of training 5

As illustrated by the mean return curve in Figure 8, the agent conquered its cowardice bias. The gain per epoch exploded, jumping from a negative stagnation to a score of +30. This evolution is naturally correlated with an increase in survival time. More counter-intuitively, the loss function also increased. This apparent paradox (a rising loss while the model improves) is specific to policy gradient methods: as the agent abandons its vegetative state to explore high-impact actions, the calculated advantages increase, while its extreme confidence (initially due to its hiding strategy) decreases, generating climbing loss values.

However, visual analysis of the agent's behavior during the inference phase (using a deterministic argmax policy to eliminate exploration stochasticity) revealed a critical flaw. While the agent excels at collecting, it never learned the abstract concept of "evasion". Instead, it converged on a distorted heuristic rule: "In the presence of a bomb, the optimal refuge is the right edge of the screen". The disastrous consequence: if a bomb falls exactly on this right edge, the agent rushes there and commits suicide.

4.2.2 Attempted Corrections: Entropy and Data Augmentation

Faced with this spatial overfitting (right-side positioning bias), several iterations were carried out to force the network to unlearn this heuristic.

The Failure of Data Augmentation (Mirror): Since the environment is perfectly symmetrical, a *data augmentation* method using a mirror effect was implemented. For each recorded interaction, its "symmetrical twin" (inverting the horizontal coordinates of the state and the

action) was added to the training batch. The theoretical objective was to mathematically destroy the "Right = Survival" correlation by proving to the network that the left side worked just as well, thus forcing it to rely on relative distances (agent-bomb) rather than absolute ones.

However, this approach ended in bitter failure (loss runaway and policy collapse). The explanation lies in the mathematical formulation of VPG. The loss function depends on the logarithm of the action's probability: $\log \pi(a|s)$. If the network, heavily biased to the right, assigns an infinitesimal probability (e.g., 10^{-6}) to a leftward action, injecting the mirrored data forces it to maximize the probability of an action it deemed impossible. The $-\log(10^{-6})$ term then generates an explosive gradient that obliterates the synaptic weights. Being an *on-policy* algorithm, VPG seems to handle the injection of data outside its own belief distribution very poorly.

Regularization via Entropy and Reward Tuning: To counter this collapse and restart exploration without distorting the distribution, an entropy term was added to the loss function, penalizing overly deterministic policies. Concurrently, the rewards were amplified once again.

Despite these adjustments, the "comfort zone" bias persisted pathologically. The agent eventually modified its rule slightly, only committing suicide on the right if the bomb fell in the right half of the screen, but still refusing to conceive of dynamic evasion based on inter-object distance.

4.2.3 Paradigm Shift: From Absolute to Relative

These successive failures led to a reassessment of the observation space. The initial hypothesis involved providing raw coordinates (X, Y) hoping the neural network would deduce spatial geometric relationships on its own (akin to human intuition). It became apparent that for a modestly sized MLP network (two layers of 64 neurons), this geometric abstraction work was not immediate (at least when subjected to tuning performed by a novice), pushing it toward absolute positional heuristics (local minima).

Faced with the network's inability to generalize evasion, the very architecture of the observation space was called into question. The classic debate between *end-to-end* learning and *feature engineering* arose. Drastically increasing the network size (e.g., 128-neuron layers) to provide it with more "cognitive resources" seemed to be a false lead, as the problem did not lie in the model's capacity, but in its lack of inductive bias.

A standard multilayer perceptron (MLP) does not possess an innate notion of spatial continuity. To deduce that a collision is imminent from absolute coordinates, it must empirically learn the conditional subtraction operation ($x_{\text{agent}} - x_{\text{bomb}} \approx 0$) for every region of the screen, which is extremely inefficient.

It was therefore decided to switch to an egocentric frame of reference. The agent no longer receives its absolute coordinates and those of the objects, but solely the relative distances $\Delta x = x_{\text{object}} - x_{\text{agent}}$. This method grants the model translation invariance: a threat located 10 units to the left will generate the same signal (-10) regardless of the agent's position on the screen. This design choice mimics the functioning of the human biological visual system and allows the network to learn a single universal rule.

4.2.4 The "Bomb Hunter" Anomaly

The implementation of relative distances caused a dazzling acceleration in learning. The agent immediately stopped systematically fleeing to the right corner, proving it had assimilated the concept of proximity. However, a paradoxical behavior persisted: evasions frequently turned into direct suicides.

Cartographic analysis of the decisions and the distribution of synaptic weights allowed for a clear diagnosis. The agent was not suffering collisions out of ignorance; it was actively targeting the bombs. The decision map revealed a behavioral inversion: facing a bomb on the left, the agent moved left; facing a bomb on the right, it moved right. It was exhibiting a sort of interception behavior, expected when collecting a mango.

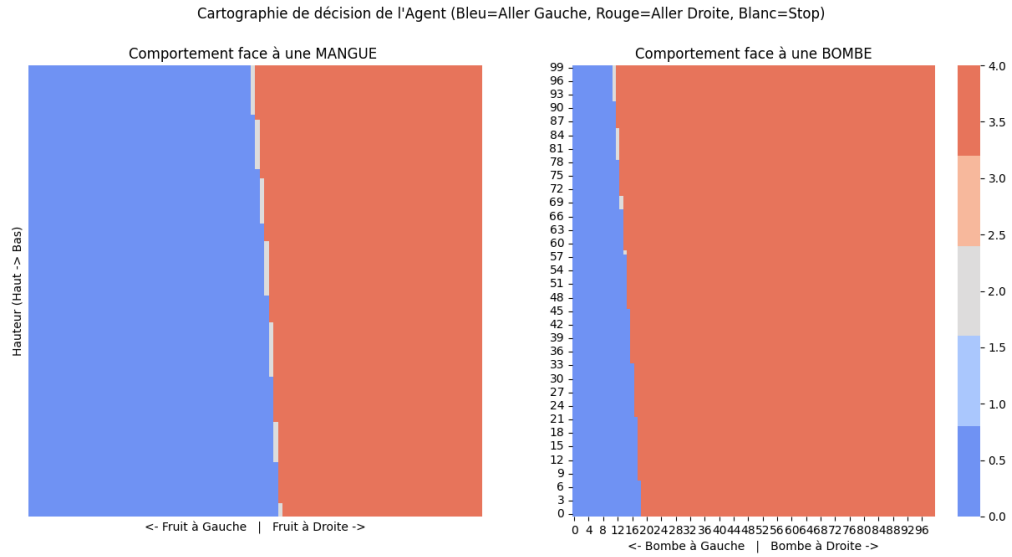


Figure 9: Decision map - inverted behavior towards bombs

Analysis of the weights in the network's first layer (see Figure 10) by calculating the mean of the weights associated with each input helped explain this phenomenon. The weight granted to the spatial variable (Δx) literally crushed the weight granted to the semantic variable (`is_bomb`). Mathematically, the geometric signal of attraction ("the object is on the right") overpowered the semantic penalty ("it is a threat"). The network had become "obsessed" with the collection movement and incapable of learning the conditional operation necessary to invert its displacement vector in the presence of a bomb.

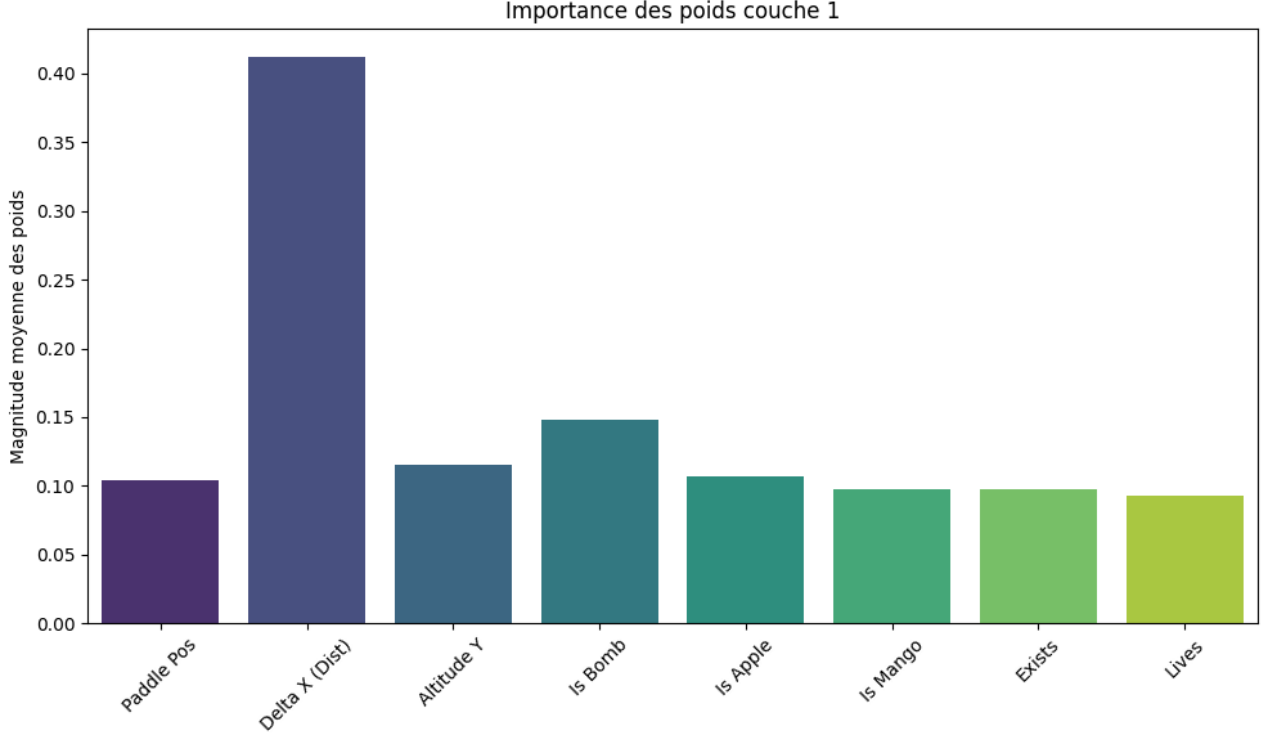


Figure 10: Synaptic analysis of layer 1

4.2.5 Separation of Streams: Synaptic Specialization

Since the network struggled to perform the internal conditional multiplication (Distance $\times$ Object Type), the observation space underwent a final structural overhaul aimed at separating the information streams.

Instead of providing a generic distance and a boolean identifier, the distances were split into two distinct channels:

- $\Delta x_{\text{attract}}$: Relative distance of a fruit (equals 0 if the object is a bomb).
- $\Delta x_{\text{repulse}}$: Relative distance of a bomb (equals 0 if the object is a fruit).

This separation successfully resolved the synaptic conflict. Each neuron in the first layer was able to specialize unequivocally: certain weights dedicated themselves exclusively to attraction (chase), and others exclusively to repulsion (flee).

Training run 14 paired the exploitation of this new 30-observation vector with the introduction of a slight continuous reward signal (dense reward of +0.045 per step) when the agent manages to align itself beneath the nearest fruit. The result was a major success. The suicidal behavioral inversion completely vanished. The agent now actively evades bombs by maintaining a safe distance, while optimizing its trajectories for fruit collection thanks to this spatial micro-guidance. Although a slight residual asymmetry remains in its movements, the resulting policy is stable, highly performant, and demonstrates a comprehensive understanding of the environment’s mechanics.

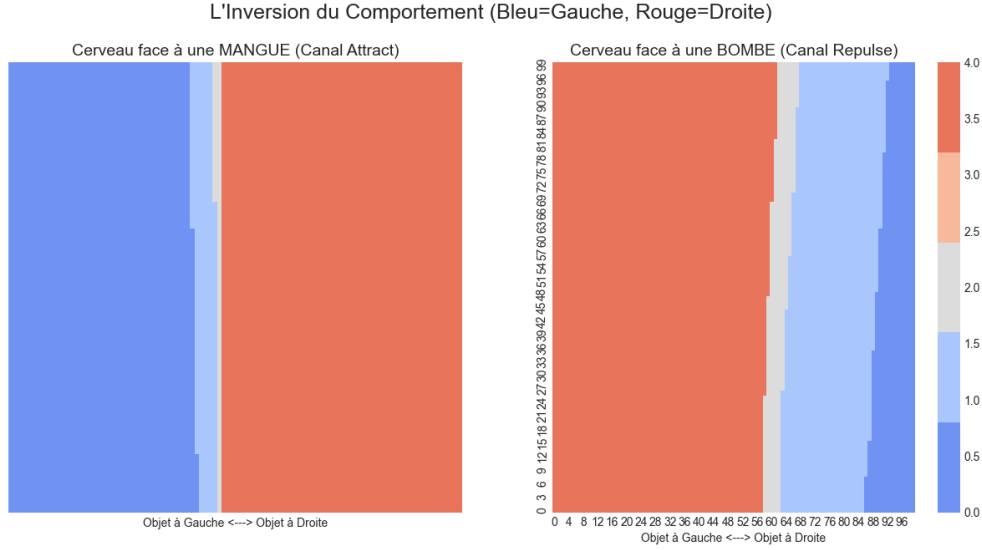


Figure 11: Decision map - Healthy behavior towards bombs

The decision map in Figure 13 shows that the suicidal behavior towards bombs has been defeated.

4.3 Phase 3: Transition to the Off-Policy Architecture: The Deep Q-Network (DQN)

The structural limitations of the Vanilla Policy Gradient (VPG), notably its low sample efficiency and sensitivity to geometric local minima, motivated the shift to a radically different architecture: the Deep Q-Network (DQN).

This paradigm shift implies profound modifications in the management of experience, the calculation of error, and the temporality of learning.

4.3.1 Fundamental Differences and Internal Mechanics

Unlike VPG, which learns from complete trajectories before destroying them (*On-Policy*), DQN learns continuously from fragmented memories.

- **Network Architecture and Memory:** The agent is now equipped with a *replay buffer* (circular memory $\mathcal{D}$) storing individual transitions $(S_t, A_t, R_t, S_{t+1}, \text{done})$. The main neural network (Q-Network parameterized by θ) no longer generates a probability distribution via a softmax, but produces an output estimating the raw value (Q-Value) of each action. A second network, the *target network* (θ^-), is introduced as a "frozen" copy of the first to stabilize learning.
- **The Learning Process (Sampling):** In VPG, a batch corresponded to the chronological succession of an entire episode. In DQN, learning is desynchronized from interaction. At regular intervals, the algorithm randomly draws a sample of N transitions (128 in our case) from the *replay buffer*. This random sampling is crucial: it breaks the temporal correlation of the data, thus preventing the network from over-learning the situation locally experienced by the agent.

- **Update and Synchronization:** The network is optimized by minimizing the mean squared error (MSE Loss) between its prediction $\hat{Y}_i = Q(S_i, A_i; \theta)$ and the temporal target Y_i calculated via the Target Network:

$$Y_i = R_i + \gamma \cdot \max_a Q(S_{i+1}, a; \theta^-)$$

To ensure smooth convergence, the weights of the Target Network are continuously updated via exponential smoothing, driven by a parameter $\tau \ll 1$.

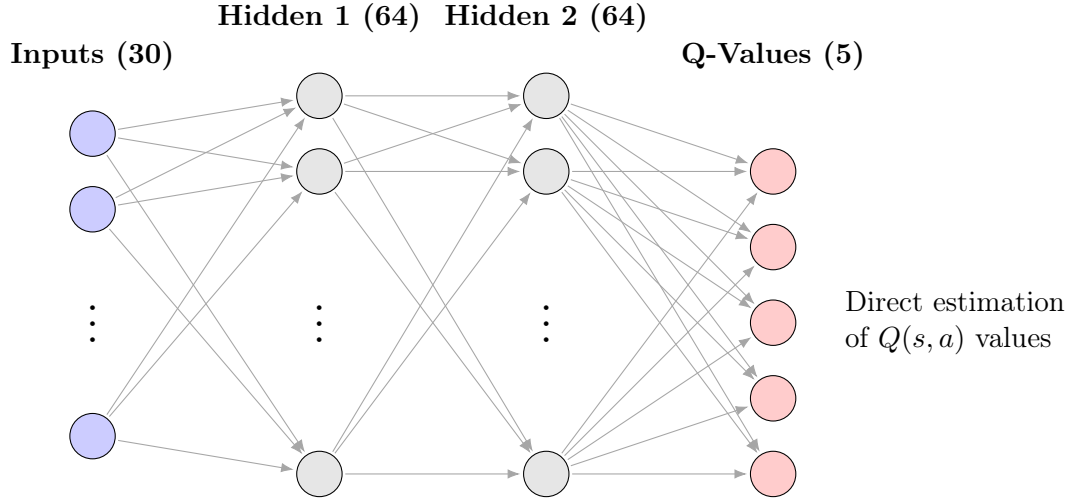


Figure 12: Architecture of the Q-Network (DQN).

4.3.2 Redefinition of Temporal Evaluation: Epochs vs Steps

The shift to DQN necessitates a semantic and technical redefinition of training temporality. In the VPG architecture, an epoch constituted a rigid structural unit (e.g., 32 episodes) marking a complete cycle of collection and learning. In DQN, learning occurs continuously on the fly. The fundamental unit is no longer the episode, but the individual time step.

This distinction is crucial for evaluation. At the beginning of training, the agent survives for only a short time (short episodes), whereas eventually, it reaches the fixed limit (long episodes). Parameterizing the training over a maximum number of episodes would expose the agent to a variable volume of data. To ensure experimental rigor, the training boundary was redefined as a total number of time steps (`TOTAL_TIMESTEPS`), thus guaranteeing a constant volume of interaction comparable to the 1,600,000 steps previously accumulated during VPG experiments. Epochs are retained merely as arbitrary display milestones (here every 10,000 steps) for metric tracking.

4.3.3 Initial Configuration of the DQN Agent

For this first off-policy iteration, the reward system was structured to include the previously validated spatial micro-guidance (+0.045/step for alignment), while maintaining strict punishments for losing objects.

Table 2: Hyperparameters and Reward Matrix (DQN - Training 1)

RL Hyperparameters		Reward Function	
Batch Size (N)	128	Mango Collected	+10
Discount Factor (γ)	0.99	Apple Collected	+7
Learning Rate (LR)	0.003	Bomb Dodged	+2
Target Soft Update (τ)	0.005	Active Alignment	+0.045/step
ϵ (Exploration)	0.9 $\rightarrow$ 0.05	Mango Missed	-5
ϵ Decay (Steps)	100 000	Apple Missed	-3
		Bomb Hit	-3

4.3.4 Behavioral Analysis of DQN: Instability and Estimation Bias

The implementation of the off-policy architecture generated a spectacular acceleration in the initial learning phase. The agent rapidly transitions from purely stochastic exploration, stemming from random weight initialization, to a more "determined" exploitation policy. However, this policy seems riddled with hard-to-explain flaws. Indeed, the agent is capable of executing coherent fruit collection sequences (long enough to collect 3 to 5 items) but unexpectedly exhibits behaviors that could be described as "aberrant" (collecting an initially distant bomb, then dodging a very close fruit). This chaotic evasion policy gives the impression that the agent's training is incomplete, even though all of its phases have already taken place. As a result, the agent is unable to achieve satisfactory scores, systematically losing the game before reaching 1000 survival steps.

Cartographic mapping of the agent's decisions when facing a bomb reveals paradoxical behaviors. Whereas the evasion policy should logically depend on the lateral distance separating the agent from the threat, we observe that decisions are almost exclusively conditioned by the bomb's altitude. Furthermore, the decision boundaries relating to mango collection appear significantly more chaotic, contrasting with the fluidity of the policies previously obtained under VPG.

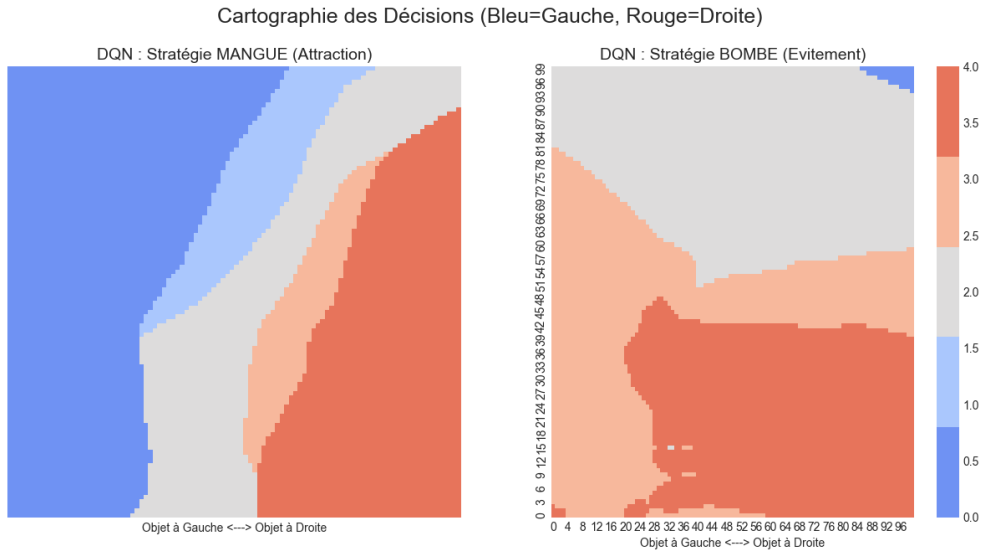
Figure 13: Decision map – neglect of the Δx factor

Figure 13 illustrates this classic phenomenon of suboptimal optimization in deep learning: *feature neglect* arising from a spurious correlation (or one whose importance is overestimated, as in our case).

4.3.5 Hyperparameter Adjustment: The Sensitivity of the Learning Rate

The lack of stable convergence during the last training run suggests an inadequate tuning of the fundamental hyperparameters. Indeed, the learning rate value had been fixed at 0.003 since the beginning of the DQN algorithm implementation.

While such a magnitude could be suited for an *on-policy* method like VPG (where gradient estimation and weight updates are smoothed over experience batches of several thousand time steps) it proves destructive in *Deep Q-Learning*.

In this *off-policy* architecture, synaptic weights are updated much more frequently (here, every 4 steps) using mini-batches sampled randomly from the *replay buffer*. Too high a learning rate coupled with this high update frequency creates violent oscillations. As soon as the agent encounters an atypical transition, the optimizer takes a disproportionate correction step, thereby destroying nascent policies and mathematically preventing the algorithm from converging towards an optimal solution.

It was therefore decided to drastically reduce the learning rate to 0.00025, which corresponds to the empirical baseline value used by DeepMind researchers in the founding DQN paper.

In parallel, a critical rebalancing of the reward function was carried out to correct a persistent behavioral anomaly: "profitable suicide." In the previous configuration, if a bomb and a fruit were falling in alignment, the agent estimated that the penalty linked to catching the bomb would be mathematically offset by the future return generated by the fruit. It would then attempt the interception, intrinsically failing to grasp that contact with the explosive constituted a terminal state (ending the episode and rendering future collection impossible).

To enforce threat evasion as an absolute priority and break any incentive for sacrifice, the collision penalty was massively increased, while the loss of a fruit was made more tolerable:

- **Survival Priority:**

- bomb_caught = -30
- bomb_dodged = 1

- **Collection Incentives:**

- mango_reward = 5
- apple_reward = 3

- **Loss Tolerance:**

- mango_loss = -1
- apple_loss = -1

5 Results and Comparative Analysis

5.0.1 Performance Benchmark

Table 3 summarizes the final performances of the best models obtained from the two algorithms:

Table 3: Comparative Survival Performances (over 20 games)		
Metric	VPG	DQN
Average convergence time	$\approx 300,000$ steps	$\approx 30,000$ steps
Level 2		
Average survival steps	3326 ± 3357	2034 ± 898
Survival record (Max steps)	12694	4166
Level 3		
Average survival steps	3757 ± 2345	1492 ± 864
Survival record (Max steps)	8235	3777

The analysis of these results highlights the fundamental trade-off between sample efficiency and learning stability. From a temporal standpoint, the DQN confirms its theoretical superiority by converging ten times faster than the VPG (approximately 30,000 steps versus 300,000).

Nevertheless, this velocity comes at the expense of the policy’s overall robustness. Across both evaluated difficulty levels, the VPG vastly outperforms the DQN in terms of survival. At level 3, the VPG agent survives on average more than twice as long (3757 steps against 1492 for the DQN) and achieves a survival record (8235 steps) demonstrating a true mastery of evasion over the long term. The DQN, on the other hand, plateaus quickly due to behavioral flaws (suicides) induced by the overestimation of its Q-values.

6 Conclusion

In conclusion, although the *Deep Q-Network* (DQN) algorithm is theoretically reputed to be more powerful and endowed with better sample efficiency than the *Vanilla Policy Gradient* (VPG), its practical implementation within this environment encountered multiple failure modes.

The *off-policy* architecture demonstrated a strong propensity to overestimate the value of certain state-action pairs, a classic phenomenon known as *value overestimation*. This instability in credit assignment led the agent to incorrectly evaluate the real risk posed by bombs. Misled by locally distorted expectations of gain, the network regularly opted for fatal trajectories, striking bombs out of overconfidence or due to the contagion of adjacent synaptic values.

Thus, despite extensive iterations of *fine-tuning*, regularization, and *reward shaping*, the Vanilla DQN failed to surpass the robustness of the VPG. The stochastic policy of the latter, although slower to converge, ultimately generated much more stable learning and a significantly safer and more performant evasion policy within the strict framework of this study.